

Validação e Sanitização de entradas

Programação para a Web 2

Aula 12

Evandro J.R. Silva

Validação no *backend*

- A validação no navegador (HTML5, JavaScript, React Hook Form, etc.) serve principalmente à **experiência do usuário**:
 - *Feedback* imediato;
 - Menos idas ao servidor;
 - Formulário mais claro.
- Mas **não** é controle de segurança, porque o cliente não é confiável
 - JavaScript pode ser desligado;
 - O HTML pode ser editado no *DevTools* (required, min, max, pattern);
 - Qualquer pessoa pode enviar a requisição com curl, Postman, Burp, um script ou outro cliente HTTP.
 - Aplicativos mobile, bots e integrações também “falam” HTTP sem passar pelo *front*.

Validação no *backend*

- O servidor é o único ponto que um *dev* controla!
- Portanto, toda decisão de aceitar, persistir ou executar algo com um dado **precisa** ser refeita lá, **antes** da lógica de negócio e **antes** de qualquer uso em SQL, arquivo, e-mail, *shell* ou HTML.
- Regra prática:

Validação no cliente = UX.

Validação no servidor = integridade + segurança

As duas convivem; só a segunda é obrigatória

Validação no *backend*

- Validação de entrada **reduz a superfície de ataque**, mas **não substitui** as defesas primárias de injeção
 - Consultas parametrizadas;
 - Encoding contextual na saída;
 - APIs seguras de arquivo, etc.

Validação no *backend*

Validar × sanitizar × escapar

Técnica	Pergunta	Ação típica	Risco se for a única defesa
Validação	“Este valor é aceitável?”	Aceitar ou rejeitar	Baixo, se for <i>allowlist</i> + tipagem forte
Sanitização	“Posso limpar isto?”	Remover/alterar caracteres	Alto: atacante contorna a “limpeza”
<i>Encoding/escaping</i>	“Como usar isto neste contexto?”	Transformar na saída (HTML, SQL param, URL)	É a defesa correta contra XSS/Injeção no ponto de uso

Validação no *backend*

- Sanitizar (por exemplo, apagar `../` de um caminho) é frágil: `abc/.....//xyz` vira `abc/../xyz` depois da limpeza. É preferível **rejeitar** o que não bate com o contrato.
- Sanitização é melhor utilizado quando o produto precisa aceitar *rich content*. Neste caso é aconselhável utilizar alguma biblioteca especializada (DOMPurify, ou o HTML *sanitizer* do *framework*).

Tipos de Validação

- A OWASP recomenda validar em dois níveis técnicos e, na prática acadêmica/profissional, convém explicitar um terceiro: o de negócio. A ordem natural é: forma → valor no contexto → regra da aplicação.
- Em outras palavras: validação sintática → validação semântica → validação de negócio.

Tipos de Validação

- **Validação sintática**

- Pergunta: o dado tem a forma esperada?
- Exemplos:
 - CPF com 11 dígitos (ainda sem checar dígito verificador).
 - E-mail com formato básico.
 - Data ISO `YYYY-MM-DD`.
 - UUID v4.
 - Inteiro no intervalo do tipo.
 - Enum: `status ∈ {aberto, fechado, cancelado}`.
 - Comprimento mínimo e máximo (`{1,80}`).
 - *Allowlist* de caracteres (`^[a-z0-9_]{3,16}$`).

Tipos de Validação

- **Validação sintática**

- Técnicas:

- Tipagem forte do *schema* (JSON Schema, Pydantic, class-validator, Bean Validation, Zod no BFF e de novo no backend).
 - *Allowlist*, não *denylist* (`<script>` em minúsculas, misto, *encoded*, etc. escapam *denylist*).
 - Regex âncorada (`^...$`), sem `.` guloso desnecessário (ReDoS).
 - Canonicalização **antes** de validar (UTF-8 decodificado uma vez; não validar o valor ofuscado)

- Falha sintática → `400 Bad Request` (ou equivalente da API), sem processar.

Tipos de Validação

- **Validação semântica**

- Pergunta: o valor, já bem formado, faz sentido neste contexto?
- Exemplos:
 - Data inicial **antes** da data final
 - Preço ≥ 0 e $\leq$ teto configurado.
 - CEP existente e pertencente ao município informado.
 - `quantidade` $\leq$ estoque disponível (consulta no servidor).
 - `id_categoria` existe na base.

Tipos de Validação

- **Validação de negócio**

- Pergunta: esta operação é permitida para este ator, neste estado, agora?
- Exemplos:
 - Aluno só pode se matricular se houver vaga **e** se o período de matrícula estiver aberto.
 - Desconto de 40% só se o cupom estiver ativo **e** o usuário não o tiver usado.
 - Transferência não pode deixar saldo negativo **e** não pode ir para a própria conta se a regra proibir.
 - Não se altera o `preço` enviado pelo cliente se o preço oficial vive no servidor (nunca confiar em preço/total vindos do *front*).

Tipos de Validação

- **Mini-exemplo**

```
POST /pedidos  
{“produtoID”: “abc”, “qtd” > 2, “precoUnitario”: 0.01}
```

- **Sintática:** produtoID UUID, qtd inteiro 1-99, precoUnitario decimal.
- **Semântica:** produto existe; qtd <= estoque.
- **Negócio:** precoUnitario é ignorado; o servidor busca o preço vigente; aplica política de desconto; recusa se o cliente estiver bloqueado

Confiança zero nas entradas do cliente

- Tudo o que atravessa a fronteira de confiança (navegador, app, outro microsserviço sem autenticação mútua forte, *webhook*, arquivo, *query string*, *header*, *cookie*, *hidden field*) é **não confiável** até ser validado no sistema em que você confia.
 - Analogia: o caixa do super-mercado não aceita que você leve o preço escrito em um papel; ele consulta o sistema.

Confiança zero nas entradas do cliente

- Algumas consequências
 - **Hidden fields e tokens de front** não são segredo. `userID`, `role=admin`, `preco=1` no HTML são sugestões, não fatos.
 - **Dados “já gravados” também podem ser hostis.** XSS armazenado, *feed* de parceiro comprometido. Classifique fontes: confiável × não confiável.
 - **Validar cedo**, na borda do componente (*controller*), e **usar tarde** com *encoding* certo no destino.
 - **Falha = rejeitar**, não “tentar aproveitar” o valor.
 - **Allowlist > denylist.** Defina o conjunto permitido.

Confiança zero nas entradas do cliente

- Um possível *pipeline*
 1. Limitar tamanho do *body* / *upload* (DoS).
 2. Decodificar charset uma vez.
 3. Tipar (*schema*).
 4. Validação sintática.
 5. Validação semântica.
 6. Autenticação + autorização.
 7. Regras de negócio / invariantes.
 8. Persistência com consultas parametrizadas.
 9. Saída com *encoding* contextual (HTML, JSON, log, e-mail).

Leituras complementares

- Leitura “obrigatória”
 - [OWASP Input Validation Cheat Sheet](#) — sintática vs semântica, *allowlist*, cliente vs servidor.
 - [OWASP Top 10 Proactive Controls 2024, C3: Validate all Input & Handle Exceptions.](#)
 - [OWASP Developer Guida, Validate All Inputs](#) (checklist de projeto).
- Leitura de aprofundamento
 - [OWASP ASVS](#) — capítulos de validação/sanitização/*encoding*.
 - [OWASP Secure Coding Practices](#) — bloco *Input validation* (“conduct all input validation on a trusted system”).
 - [CWE-20 Improper Input Validation](#).
 - [NCSC \(UK\), Secuting HTTP-based APIs](#) — *Input validation* (sintática/semântica em APIs).
 - [GitHub Security Blog, Validate all the things](#) — validação vs sanitização e por que o cliente não conta.